

Comunidad Python · Sesión 12

BLOQUE 12

Programación Orientada a Objetos

POO en Python

Modelá el mundo real con código

Lo que ya dominamos

(x) Variables y tipos

🔗 Condicionales

↺ Bucles

☰ Listas y diccionarios

ƒ Funciones

[] Comprensiones



¡Base sólida!

Con estos bloques ya estás listo para dar el siguiente salto.

¿Qué es la POO?



Organizar el mundo real

como plantillas y ejemplos

Un paradigma que organiza el código en **clases** y **objetos** — modelando entidades del mundo real con sus datos y comportamientos.



Reutilizable

Escribe una vez, usá muchas veces



Organizado

Código modular y mantenible



Escalable

Crece con tu proyecto

Clases y Objetos

CLASE

Perro

 Nombre Raza Edad ladrar() correr()*Plantilla / Molde*

instancia

*Instancias concretas***OBJETO**

perro1

Rex

Labrador

3 años

OBJETO

perro2

Luna

Beagle

1 año

OBJETO

perro3

Max

Boxer

5 años

Atributos y Métodos

Atributos

Los datos del objeto

 `nombre = 'Rex'`

 `raza = 'Labrador'`

 `edad = 3`

Describen el estado del objeto

Métodos

Las acciones del objeto

 `ladrar()`

 `correr()`

 `comer()`

Definen el comportamiento



El método especial

`__init__`

Se ejecuta automáticamente al crear un objeto. Es la 'receta de nacimiento'.

1

Creás el objeto

```
perro1 = Perro(...)
```



2

Python llama a `__init__`

Automáticamente, sin que lo pidas



3

El objeto está listo

Con todos sus atributos inicializados

Encapsulación

Interfaz Pública

Lo que el mundo puede ver



Datos Internos

Protegidos

Ocultar los detalles internos y exponer solo lo necesario — como una cápsula que protege su contenido.

👁️ Público

Accesible desde afuera

```
nombre, ladrar()
```

🚫 Privado

Solo accesible internamente

```
_salud, __clave
```

PILARES DE POO

Herencia

Una clase hija **hereda** atributos y métodos de una clase padre — y puede agregar los suyos propios.



Polimorfismo y Métodos Especiales

Polimorfismo

El mismo método, comportamientos distintos según el objeto.

Perro → `hacer_sonido()` → *¡Guau!*

Gato → `hacer_sonido()` → *¡Miau!*

Pez → `hacer_sonido()` → ... 

Métodos Especiales

Los 'dunder methods' (doble guión bajo) le dan superpoderes a tus clases.

`__str__` Representación en texto

`__len__` Permite usar `len()`

`__eq__` Comparación con `==`

Lo que aprendimos hoy

 Clases y Objetos

 Atributos y Métodos

 Constructor `__init__`

 Encapsulación

 Herencia

 Polimorfismo

☆ ¡Ya tenés las herramientas para pensar en objetos!

Próximos Pasos

 Practicar creando tus propias clases

 Sesión 13: Ejercicios POO en proyectos reales

 Leer: docs.python.org — clases